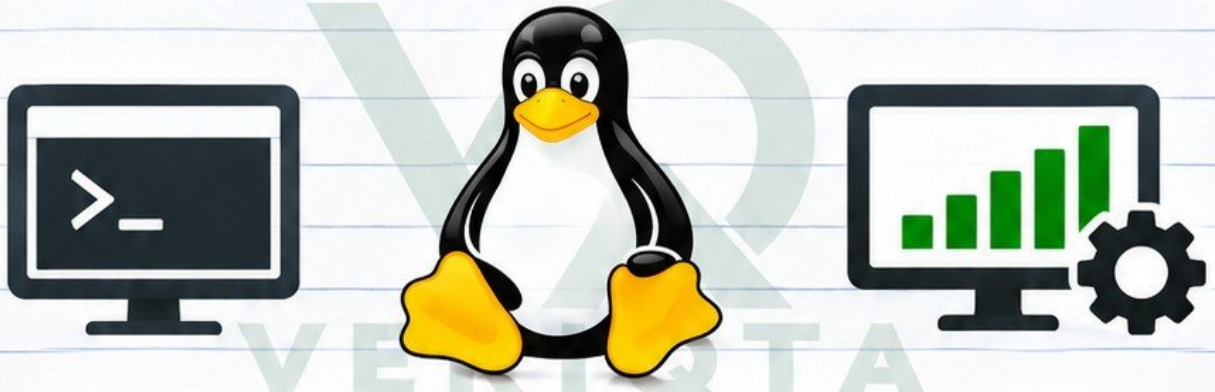
VERIQTA

The Linux Performance Tuning Handbook

Production Diagnostics, Kernel Tuning, and Real Performance Fixes



20 Real-World Scenarios to Diagnose, Optimize, and Fix

- CPU & Scheduling
- Memory & NUMA
- Storage & Filesystem
- Network & TCP Tuning
- Kernel Parameters
- Performance Diagnostics
- eBPF Tracing
- Observability & Metrics
- Real Fixes & Trade-offs
- Production Tuning

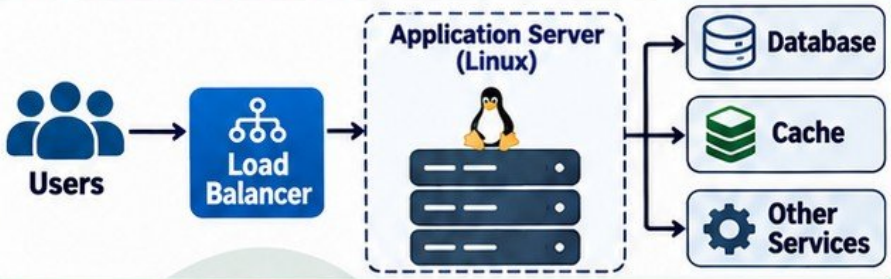
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

The Server Is Slow, but CPU, RAM, and Disk Look Healthy

Key Objectives

- Investigate latency when conventional dashboards show no saturation.
- Distinguish resource utilization, saturation, throughput, and response time.
- Correlate vmstat, mpstat, pidstat, sar, and PSI metrics.
- Identify runnable queues, blocked tasks, and hidden contention.

System Overview



Key Concepts

Utilization
How much of a resource is being used (e.g., CPU %).

Saturation
Whether the resource is a bottleneck (queues, waits).

Throughput
Amount of work completed per unit time (e.g., requests/s).

Response Time
Time to serve a request (e.g., p50, p95, p99 latency).

Essential Commands and What to Look For

Command	Purpose	Key Signals to Check
`vmstat 1 5`	System-wide CPU, memory, and I/O status.	r (runnable queue), b (blocked), wa (I/O wait).
`mpstat -P ALL 1 5`	Per-core CPU usage and pressure.	Uneven core usage, high %sys, or softirq.
`pidstat -w 1 5`	Per-process CPU, I/O, and context switches.	High voluntary/involuntary context switches.
`sar -u -q -d 1 5`	Historical view of CPU, run queue, and disk.	Run queue growth, I/O wait, or spikes over time.
`cat /proc/pressure/*`	PSI (Pressure Stall Information) for CPU, memory, and I/O.	High some/total stall values (e.g., cpu.some > 10%).

Diagnosis Flow

Check latency (p99) → Check run queue (vmstat) → Check per-core load (mpstat) → Check process waits (pidstat) → Confirm pressure (PSI)

Production Challenge

Diagnose an application with normal CPU usage but worsening p99 latency.

Senior Takeaway

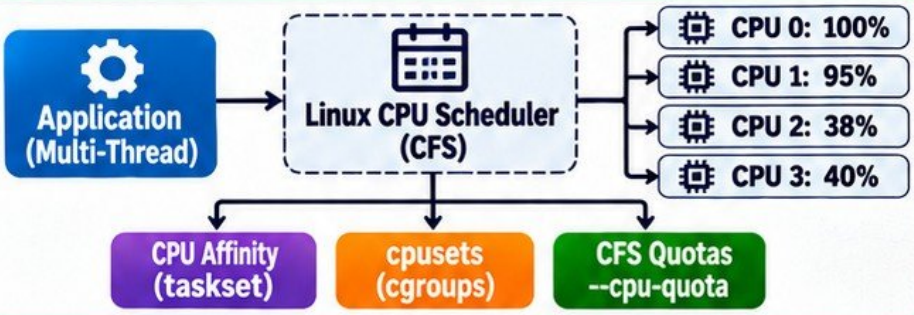
A healthy-looking CPU, RAM, and disk does not mean a healthy system. Always investigate queues, contention, and resource pressure before tuning.

The CPU Is 40% Idle, but Your Application Cannot Get Scheduled

Key Objectives

- Investigate CPU run queues, scheduler latency, and per-core saturation.
- Examine CPU affinity, cpusets, CFS bandwidth throttling, and container quotas.
- Use perf sched, /proc/schedstat, pidstat -w, and cgroup CPU statistics.
- Explain why average CPU utilization hides overloaded individual cores.

Architecture and Concepts



Key Concepts

Run Queue
Tasks waiting for CPU time (r value).

Scheduler Latency
Time a task waits before it runs.

CPU Affinity
Restricts a process to specific CPU cores.

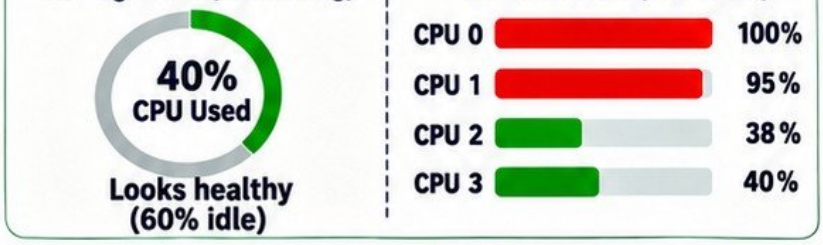
CFS Bandwidth
Limits CPU time using quotas and periods.

Container Quota
CPU limit for containers (cgroups).

Essential Commands and What to Look For

Command	Purpose	Key Signals to Check
`mpstat -P ALL 1 5`	Per-core CPU utilization.	One or more cores high (e.g., 95–100%) while overall CPU is moderate (e.g., 40%).
`pidstat -w 1 5`	Task switching and CPU wait (sched delay).	High CS (context switches) or high wait time.
`perf sched timehist`	Analyze scheduling delays for tasks.	Large sched delays (e.g., > 10–50 ms).
`cat /proc/schedstat`	Detailed scheduler statistics per CPU.	High run_queue and wait times on specific CPUs.
`cat /sys/fs/cgroup/cpu*/cpu.stat`	CPU usage, quotas, and throttling (cgroups).	nr_throttled and throttled_time increasing.

Why Average CPU Can Be Misleading



Senior Decision

Change CPU placement, quotas, or application concurrency only after identifying the scheduling bottleneck with evidence.

Key Takeaway

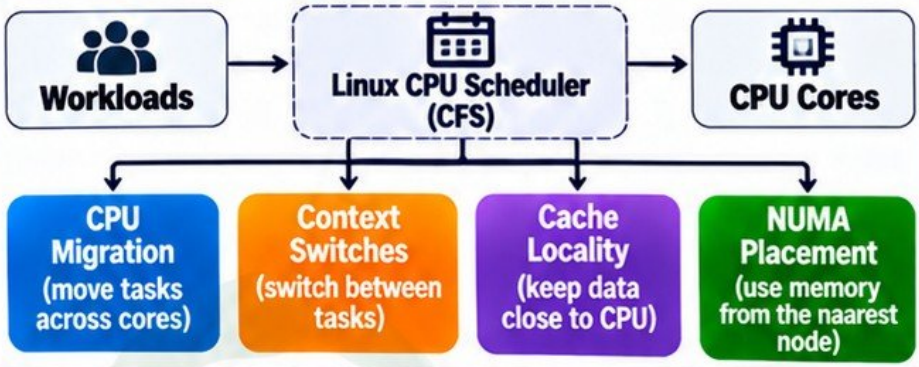
Total CPU utilization can be low while one or more cores are saturated, causing high scheduling delays and application latency.

The Linux Scheduler Is Fighting Your Workload

Key Objectives

- Examine CPU migrations, context switches, cache locality, and NUMA placement.
- Compare scheduling behavior for interactive services, batch processing, and CPU-intensive workloads.
- Investigate taskset, chrt, nice, numactl, and scheduler tracing.
- Identify when priority changes create starvation or unpredictable latency.

Linux Scheduling Overview



Scheduling Behavior by Workload

Workload Type	Scheduler Focus	Typical Settings / Notes
Interactive (e.g., API, Web)	Low latency and quick response.	Normal priority (nice 0), avoid CPU starvation.
Batch (e.g., ETL, Reports)	Throughput over latency.	Lower priority (nice +10), run in off-peak windows.
CPU-Intensive (e.g., Build, Encode)	Maximize CPU usage, minimize switches.	CPU affinity (taskset), consider NUMA (numactl).

Essential Commands

Command	Purpose
`taskset -c 0-3 <cmd>`	Pin a process to specific CPU cores (CPU affinity).
`chrt -f 50 <cmd>`	Run with real-time scheduling and priority.
`nice -n 10 <cmd>`	Run with a lower (or higher) scheduling priority.
`numactl --cpunodebind=0 --membind=0 <cmd>`	Control CPU and memory placement on a NUMA node.
`perf sched timehist`	Trace scheduling events, delays, and migrations.
`pidstat -w 1 5`	Show per-process context switches and wait time.

Key Observations to Check

- High context switches (cs) may indicate contention or too many threads.
- Frequent CPU migrations reduce cache efficiency and increase latency.
- Remote NUMA memory access increases latency and lowers throughput.
- High-priority tasks (e.g., chrt -f) can starve normal workloads if misused.

Production Challenge

Improve application latency by optimizing CPU placement and scheduling without unfairly starving other workloads.

Senior Decision

Use CPU affinity, priority, and NUMA tuning only after confirming scheduler delays, migrations, or contention as the real bottleneck.

The Performance Regression That Came from CPU Frequency and Firmware

Key Objectives

- Examine CPU frequency scaling, turbo behavior, thermal throttling, and power limits.
- Investigate CPU governors, turbostat, cpupower, and thermal sensors.
- Distinguish guest-visible CPU metrics from hypervisor and hardware effects.
- Analyze why identical deployments perform differently across instance families.

CPU Frequency and Performance Flow



Key CPU Concepts

CPU Governor
Controls how CPU frequency scales (e.g., performance, ondemand).

Turbo Boost
Temporarily increases CPU speed above base frequency when possible.

Thermal Throttling
Reduces CPU speed to prevent overheating (based on temperature).

Power Limits
Firmware or platform limits that cap CPU performance (e.g., TDP).

Essential Commands, Purpose, and Example

Command	Purpose	Example Output (Sample)
`cpupower frequency-info`	Show CPU governor and frequency range.	governor: ondemand available: 1.20 GHz – 3.40 GHz
`cpupower frequency-set -g performance`	Set a fixed high-performance governor.	Setting governor to "performance" ... OK
`turbostat`	View real-time CPU frequency and turbo status.	Avg_MHz: 3298 Busy%: 92.1 PkgTmp: 71C
`cat /sys/class/thermal/thermal_zone*/temp`	Read CPU temperature (in millidegree Celsius).	72000 (72°C)
`lscpu`	Check CPU model and supported features.	Model name: Intel(R) Xeon(R)... CPU MHz: 1200.000 – 3400.000

Why Identical Deployments Perform Differently

Factor	Impact on Performance
CPU Instance Family	Different base/turbo speeds and power limits.
Firmware / BIOS	Power and thermal policies can cap performance.
Virtualization Layer	vCPU scheduling and noisy neighbors (cloud).
Thermal Environment	Higher temperatures trigger throttling.
Governor Settings	May keep CPU at lower frequency during load.

Senior Judgment

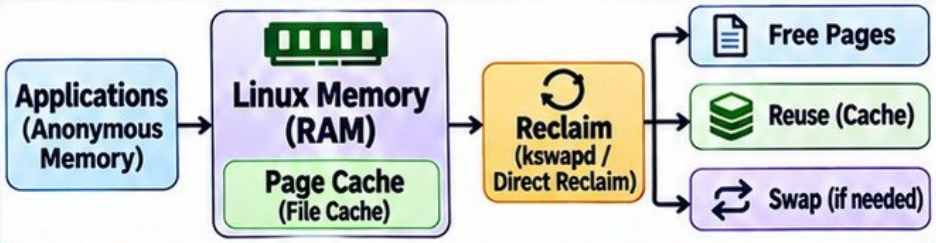
Benchmark sustained workload performance (e.g., 10–30 minutes) rather than relying on advertised CPU specifications or short burst results.

The Server Has Free Memory, but Linux Is Reclaiming Pages Aggressively

Key Objectives

- Understand anonymous memory, page cache, reclaim, and working-set behavior.
- Investigate kswapd, direct reclaim, major faults, and memory PSI.
- Examine /proc/meminfo, /proc/vmstat, and slabtop.
- Identify when reclaim stalls create latency without triggering the OOM killer.
- Separate memory pressure from an actual application memory leak.

Linux Memory and Reclaim Flow



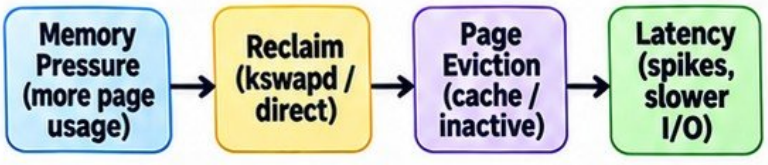
Key Memory Concepts

Anonymous Memory	Page Cache	Reclaim	Working Set	PSI (Pressure Stall)
Process memory (heap, stack) not backed by files.	Cached file data in RAM to speed up I/O.	Frees memory by evicting cache or inactive pages.	The active memory a workload needs to run efficiently.	Shows when CPU, memory, or I/O is being stalled due to pressure.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example Signal
`cat /proc/meminfo`	View memory breakdown.	MemFree, Cached, Active, Inactive, Slab.
`cat /proc/vmstat`	Check reclaim and fault activity.	pgscan_*, pgsteal_*, pgmajfault increasing.
`slabtop`	Inspect kernel slab memory usage.	Large, growing caches (e.g., dentry, inode).
`cat /proc/pressure/memory`	Memory PSI (stall time).	some/total > 0 (e.g., some avg10=5.20%).
`pidstat -r 1 5`	Per-process memory and faults.	Major faults (majflt/s) spikes.
`ps -eo pid,comm,rss,vsz --sort=-rss | head`	Top memory consumers.	Identify growth vs stable usage.

Reclaim and Latency: How It Happens



Production Challenge

Determine whether performance degradation is caused by memory pressure and reclaim or by a real application memory leak.

Senior Judgment

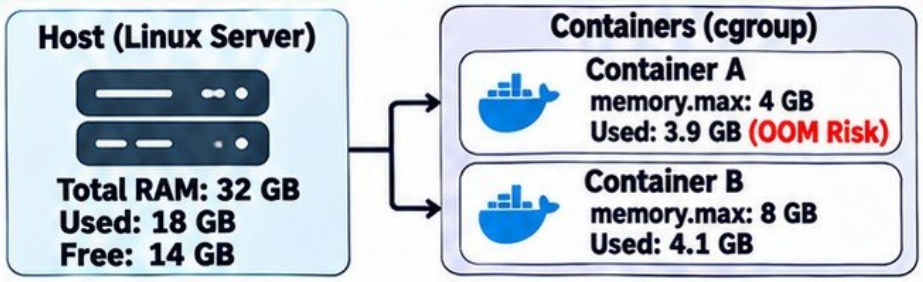
High reclaim and PSI with stable RSS usually indicate memory pressure (not a leak). A real memory leak shows continuously growing RSS with no decay over time.

Your Application Is Running Out of Memory Without Running Out of RAM

Key Objectives

- Investigate cgroup memory limits, memory.high, memory.max, and OOM events.
- Compare container memory accounting with host-level available memory.
- Examine page cache charging, memory throttling, and reclaim behavior.
- Distinguish kernel OOM kills from cgroup-local OOM kills.
- Determine whether to increase limits, reduce working-set size, or correct application behavior.

Host vs Container Memory Accounting



Key Concepts

cgroup Memory
Linux control group that limits and accounts memory per container.

memory.high
Soft limit; triggers throttling and reclaim before hard limit.

memory.max
Hard limit; maximum memory a cgroup can use (OOM if exceeded).

Page Cache Charging
File cache can count toward a container's memory limit.

OOM Events
Killing of processes when memory limits are exceeded (kernel or cgroup).

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`cat /sys/fs/cgroup/<id>/memory.max`	View the hard memory limit.	9223372036854771712 (no limit) or 4294967296 (4 GB)
`cat /sys/fs/cgroup/<id>/memory.high`	View the soft memory limit.	2147483648 (2 GB)
`cat /sys/fs/cgroup/<id>/memory.current`	Check current memory usage.	4080216064 (≈ 3.8 GB)
`cat /sys/fs/cgroup/<id>/memory.events`	See throttle/OOM events.	high 234 oom 0 oom_kill 1
`docker stats (or ctr + crictl)`	Container memory usage.	MEM USAGE / LIMIT 3.8GiB / 4GiB (95%)
`dmesg | grep -i "oom"`	Find kernel OOM kill events.	[OOM] Killed process 1234 (app) ...
`journalctl -u kubelet | grep -i oom`	Find container OOM (Kubernetes).	Container marked OOMKilled

OOM Types: Know the Difference

Kernel OOM Kill

- Triggered by system-wide memory exhaustion.
- Affects any process on the host.
- Message in dmesg: "Out of memory: Killed process ..."

cgroup-local OOM Kill

- Triggered when a container exceeds memory.max.
- Only affects processes in that cgroup/container.
- Shown in cgroup memory.events (e.g., oom_kill 1).

Decision Flow: What to Do Next

Check cgroup limits (max/high) → Verify memory usage (current) → Check OOM events (memory.events) → Decide: Increase limit / reduce working set / fix app

Senior Decision

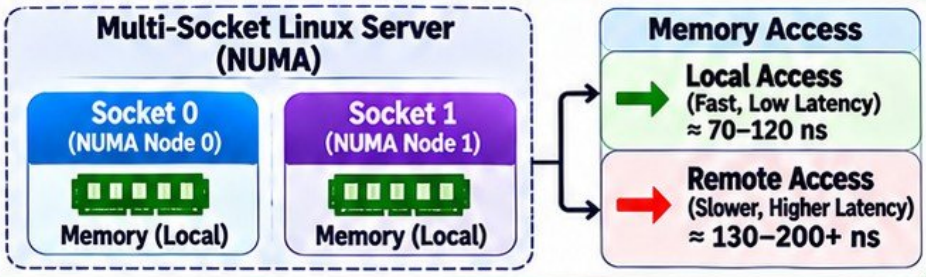
Increase memory limits only if the workload legitimately needs more memory. Otherwise, reduce the working set (e.g., caching, heap size) or fix inefficient memory usage in the application.

NUMA Locality Is Quietly Destroying Database Performance

Key Objectives

- Understand local versus remote memory access and NUMA node imbalance.
- Investigate numastat, numactl, memory placement, and CPU pinning.
- Examine database thread placement and memory allocation patterns.
- Evaluate automatic NUMA balancing and its potential overhead.
- Diagnose inconsistent database latency on a multi-socket server.

NUMA Architecture and Data Access



Key NUMA Concepts

NUMA Node
A socket with its own local memory (e.g., Node 0, Node 1).

Local vs Remote Access
Local = faster and higher bandwidth; Remote = slower and higher latency.

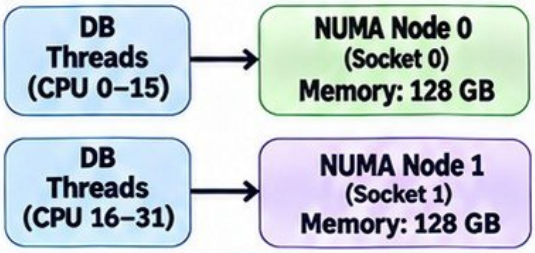
NUMA Balancing
Kernel feature that can migrate pages and tasks (d may add overhead).

CPU Pinning
Bind database threads to specific CPUs and memory nodes.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`numastat`	Show per-node memory and access stats.	Look for high "other" (remote) access (e.g., node0: 90G, other: 30G).
`numactl --hardware`	View NUMA topology (nodes, CPUs, memory).	NUMA node(s): 0,1; CPUs per node; memory per node.
`numactl --cpubind=0 --membind=0 <cmd>`	Run a process on a specific NUMA node.	Run DB on node 0 (useful for testing locality).
`ps -eLo pid,cmd,psr,ni`	Check thread-to-CPU placement.	Confirm DB threads stay on intended CPUs.
`cat /proc/sys/kernel/numa_balancing`	Check NUMA balancing status.	0 = off, 1 = on (evaluate performance impact).

Database Workload Placement Example



Production Challenge

Diagnose and fix inconsistent database latency caused by NUMA imbalance on a multi-socket server.

Senior Decision

Keep database threads and memory on the same NUMA node, and only use automatic NUMA balancing if it improves performance without adding overhead.

Key Takeaway

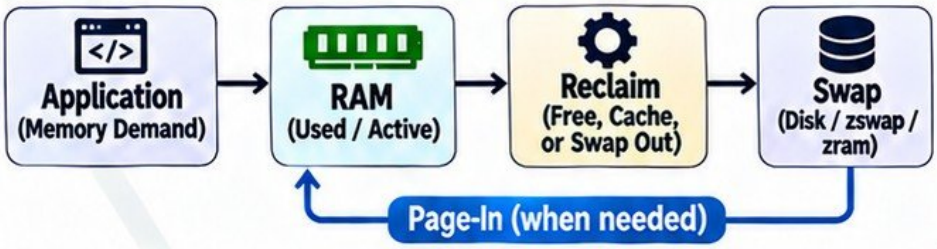
NUMA, not CPU or RAM usage, can be the hidden cause of database latency. Keep data, threads, and memory local and verify with numastat and real workload tests.

Swap Is Not the Enemy. Uncontrolled Memory Pressure Is

Key Objectives

- Examine swap activity, page-in latency, thrashing, and swap-backed memory.
- Compare swap devices, zswap, and zram where supported.
- Evaluate vm.swappiness, reclaim behavior, and workload requirements.
- Identify when disabling swap causes worse failures than controlled swapping.
- Choose swap policy based on measured latency and failure tolerance.

Memory, Reclaim, and Swap Flow



Swap Options and When to Use Them

Swap (Disk)

- Traditional swap space.
- Persistent and simple.
- Slower (disk I/O latency).
- Good for general-purpose servers.

zswap

- Compressed swap in RAM (with disk backing).
- Reduces disk I/O.
- Lower latency than disk swap.
- Good balance for most workloads.

zram

- Compressed RAM used as swap (no disk I/O).
- Fastest swap option.
- Useful for memory-constrained or ephemeral workloads.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`swapon --show`	Show active swap devices.	/dev/sda2 type=partition size=8G used=2.1G
`free -h`	Check RAM and swap usage.	Swap: 8G Used: 2.1G Free: 5.9G
`vmstat 1 5`	View swap in/out and reclaim activity.	si/so > 0 (e.g., si=204 so=188) = swapping active
`cat /proc/meminfo`	Detailed memory and swap info.	SwapTotal: 8388608 kB SwapFree: 6156288 kB
`cat /proc/vmstat | grep -E "pswpin|pswpout"`	Total swap-in/out counts.	pswpin 125430 pswpout 118760
`sysctl vm.swappiness`	Check current swappiness value.	vm.swappiness = 60

vm.swappiness Guide

Value	Typical Use
0 – 10	Keep most in RAM (e.g., DB, low latency).
20 – 40	Balanced (general servers).
60 (default)	Allow reclaim and swap when needed.
80 – 100	More aggressive swap (memory scarce or batch workloads).

Key Observations

- High si/so and sustained swap I/O indicates memory pressure.
- Page-in latency increases application response time.
- Thrashing = frequent swap in/out with high disk I/O and latency.
- Swap usage alone is not a failure if the system remains stable.

Senior Judgment

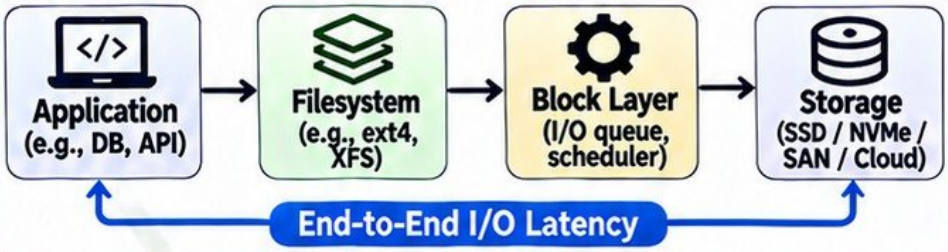
Do not disable swap by default. Use it aa safety buffer and control its behavior (e.g., vm.swappiness, zswap, or zram) based on workload, latency targets, and the cost of out-of-memory failure.

Your Disk Is Fast, but Application I/O Is Still Slow

Key Objectives

- Distinguish IOPS, throughput, queue depth, service time, and end-to-end latency.
- Investigate iostat -x, pidstat -d, iotop, and block-layer tracing.
- Examine synchronous writes, flushes, and application-generated I/O patterns.
- Explain why %util alone is unreliable as a saturation indicator.
- Identify whether the bottleneck is the application, filesystem, block layer, or storage backend.

I/O Flow and Where Latency Happens



Key I/O Concepts

IOPS
Number of I/O operations per second (reads + writes).

Throughput
Amount of data transferred per second (e.g., MB/s, GB/s).

Queue Depth (QD)
Number of outstanding I/O requests in the device queue.

Service Time (svctm)
Time the device spends processing each I/O request.

Latency
Total time from the application request to I/O completion (end-to-end).

Essential Commands and What to Look For

Command	Purpose	Key Metrics / Example
`iostat -x 1 5`	Device-level I/O stats and saturation.	r/s, w/s, rMB/s, wMB/s, await, svctm, aqu-sz, %util
`pidstat -d 1`	Per-process I/O (who is doing the I/O).	kB_rd/s, kB_wr/s, io/s per PID
`iotop -oPa`	Live view of top I/O-consuming processes.	READ/WRITE (kB/s) by process
`blktrace / btt / blkparse`	Trace block-layer I/O in detail.	I/O request timing, queueing, and completion
`cat /sys/block/<dev>/queue/*`	Check device queue and scheduler settings.	nr_requests, scheduler, read_ahead_kb, etc.

Why %util Alone Can Mislead

- %util shows the time the device was busy, not whether it i.a bottleneck.
- Modern SSD/NVMe can handle high IOPS with low %util.
- Low %util can still have high latency due to queueing, flushes, or application behavior.

Common I/O Bottlenecks

Layer	Examples
Application	Sync writes, frequent fsync(), small random I/O.
Filesystem	Journaling, metadata updates, unfavorable mount options.
Block Layer	High queue depth, scheduler overhead, writeback.
Storage Backend	Limited IOPS/throughput, networked storage (SAN/Cloud), or backend congestion.

Production Challenge

Identify the true I/O bottleneck: application, filesystem, block layer, or storage backend using metrics, traces, and controlled tests.

Senior Judgment

Use a holistic view (application → filesystem → block layer → storage) and latency metrics (e.g., await, p95/p99) to make the right tuning decision.

The Filesystem Is Turning Small Writes into Expensive Operations

Key Objectives

- Examine journaling, metadata updates, fsync, and write amplification.
- Compare ext4 and XFS behavior for relevant workloads.
- Investigate dirty-page writeback, inode pressure, and directory operations.
- Use filefrag, findmnt, strace, and filesystem statistics.
- Tune write behavior without weakening required durability guarantees.

Small Write: What Really Happens

Small Write (e.g., update a few KB) → Journaling (log the change) → Metadata Update (inode, size, timestamps) → Writeback / fsync (flush to disk)

Result: One small write can trigger multiple I/O operations (journal + metadata + data), increasing latency and I/O cost.

Key Concepts

Journaling	Metadata	fsync	Write Amplification	Dirty Pages
Records filesystem changes for crash recovery.	Information about files (e.g., inode, permissions, size, timestamps).	Forces data and metadata to be permanently written to disk.	More disk I/O than the original write (e.g., small write → multiple writes).	Modified data in RAM that will be written to disk later (writeback).

Essential Commands and What to Look For

Command	Purpose	What to Check / Example Signal
`filefrag -v /path/file`	Check file fragmentation.	Number of extents (extents > 1 = fragmented).
`findmnt -no SOURCE,FSTYPE,OPTIONS /path`	Show filesystem type and mount options.	ext4 or xfs, options (e.g., data=ordered).
`strace -e fsync,fdatasync -p <PID>`	See when the app calls fsync.	Frequent fsync() calls (e.g., many per second).
`cat /proc/meminfo | grep Dirty`	Check dirty-page (writeback) activity.	Dirty: 512000 kB (growing rapidly).
`cat /proc/sys/vm/dirty_*`	View dirty-page writeback thresholds.	dirty_ratio, dirty_background_ratio values.
`iostat -x 1`	Observe disk I/O and queue behavior.	High await or avgqu-sz during write bursts.

ext4 vs XFS (Operational View)

Aspect	ext4	XFS
Journaling	Yes (data=ordered by default)	Yes (metadata journaling, scalable)
Small Writes	Good, simple, low overhead	Typically better at high concurrency
Large/Parallel I/O	Good	Excellent (high throughput)
Best Fit	General purpose, smaller systems	Large files, high I/O, concurrent workloads

Common Cost Drivers

- Journaling every transaction (more writes).
- Frequent fsync calls (forces immediate disk writes).
- Metadata-heavy operations (create/delete/rename, many files).
- Dirty-page writeback in large bursts (I/O spikes).
- File fragmentation (more I/O to read/write data).

Production Challenge

Diagnose and reduce the overhead from journaling, metadata updates, and fsync causing high write latency for an I/O-intensive application.

Senior Decision

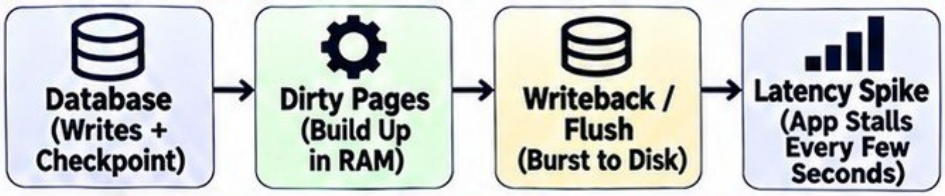
Optimize write behavior (e.g., batching writes, appropriate mount options, and writeback tuning) while still preserving the required durability and crash-recovery guarantees.

The Database Stalls Every Few Seconds, but Storage Benchmarks Pass

Key Objectives

- Investigate periodic writeback, checkpoints, flush storms, and latency spikes.
- Examine vm.dirty_background_bytes, vm.dirty_bytes, and writeback activity.
- Correlate database checkpoints with filesystem and block-device metrics.
- Explain why synthetic sequential benchmarks may miss production write patterns.
- Identify tuning options to reduce write stalls without losing durability.

What's Happening (Typical Pattern)



Pattern: Writes accumulate, then a large writeback/flush (checkpoint) causes burst I/O and temporary stalls.

Key Concepts

Checkpoint — Periodically flushes in-memory database changes to disk.

Writeback — Kernel process that writes dirty pages from RAM to the filesystem.

Dirty Pages — Modified pages in RAM that must be written to disk eventually.

vm.dirty_* — Controls how much dirty data can build up before writeback.

Flush Storm — A burst of synchronized writes (e.g., checkpoint + writeback) that increases latency.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`sysctl vm.dirty_*`	View dirty-page writeback settings.	vm.dirty_bytes=536870912 (512 MB)
`cat /proc/meminfo | grep Dirty`	Check current amount of dirty pages.	Dirty: 420000 kB (≈ 410 MB)
`cat /proc/vmstat | egrep 'writeback|dirty'`	See writeback and dirty-page activity.	nr_dirty, nr_writeback, nr_writeback_temp
`iostat -x 1`	Observe disk write activity and latency.	await, w/s, %util during stalls (e.g., await ↑)
`pidstat -d 1 -p <db_pid>`	Check database write I/O in real time.	kB_wr/s spikes during checkpoint
`strace -p <db_pid> -e fdatasync,fsync`	See when the database issues flushes.	fdatasync(...) = 0 (repeated during checkpoint)

Why Benchmarks Can Be Misleading

- Synthetic tests (e.g., sequential writes with fio) use simple, steady patterns.
- Real databases generate mixed, bursty workloads (small writes, fsync, metadata).
- Benchmarks often avoid checkpoints, background writeback, and real latency churn.

Tuning Levers (with Care)

- Adjust dirty thresholds: vm.dirty_bytes and vm.dirty_background_bytes.
- Tune database checkpoint frequency (e.g., checkpoint_timeout, max_wal_size).
- Use batched or async writes where supported.
- Monitor p95/p99 write latency and writeback activity after changes.

Production Challenge

Reduce p99 write latency by controlling dirty-page buildup and checkpoint impact while preserving crash consistency.

Senior Decision

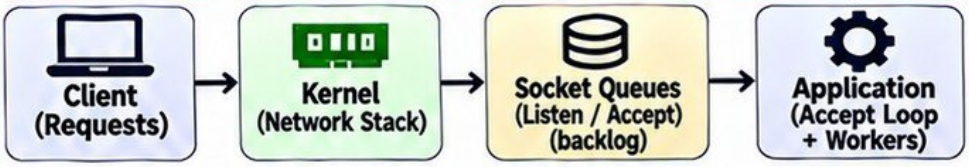
Balance database checkpoint settings and kernel writeback parameters to smooth write bursts, keep latency predictable, and maintain durability and crash recovery guarantees.

The Network Has Bandwidth, but Requests Are Queueing Inside Linux

Key Objectives

- Investigate socket backlogs, accept queues, retransmissions, and packet drops.
- Examine ss -tinm, nstat, sar -n, and /proc/net/netstat.
- Distinguish network congestion from application accept-loop saturation.
- Evaluate connection reuse, listen backlogs, and socket buffer pressure.
- Identify the real bottleneck and tune safely without hiding application slowness.

Request Flow and Where Queueing Happens



Requests can queue inside Linux (listen/accept queues, socket buffers) even when network bandwidth is available.

Key Concepts

Sacket Backlog
Pending connections waiting to be accepted by the application.

Accept Queue
Connections already established but not yet handled by the app.

Retransmission
Packets resent due to loss, delay, or lack of acknowledgment.

Packet Drops
Packets discarded (e.g,, queue overflow, buffer limits, or congestion).

Socket Buffers
Kernet memory that holds inbound/outbound data (e.g., rmem, wmem) for each socket.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`ss -tinm`	Inspect socket states, queues, and memory.	ListenRec, ListenOverflows, Recv-Q, Send-Q.
`nstat`	View key network stack counters.	TcpExt TCPListenDrops, TCPListenOverflows, RetransSegs.
`sar -n TCP,ETCP 1`	Monitor TCP activity and errors in real time.	retrans/s, active/s, passive/s, drops.
`cat /proc/net/netstat`	Detailed kernel network statistics.	TCP: ListenOverflows, ListenDrops, InSegs, OutSegs.
`sysctl net.core.somaxconn`	Check max listen backlog limit.	net.core.somaxconn = 4096 (example).
`sysctl net.ipv4.tcp_max_syn_backlog`	Check SYN backlog (pending handshakes).	net.ipv4.tcp_max_syn_backlog = 8192 (example).

Common Indicators and Meaning

Indicator	Meaning
ListenOverflows ↑	Listen queue is full (backlog too small or app not accepting fast enough).
ListenDrops ↑	Incoming connections dropped due to queue limits.
RetransSegs ↑	Packet loss, congestion, or delayed application reads.
High Recv-Q	App is not reading data fast enough (socket buffer pressure).
Low CPU + High Queues	Likely application accept-loop or I/O bottleneck, not network bandwidth.

Troubleshooting Focus

- Check listen and accept queue usage (ss -tinm, nstat).
- Correlate spikes in retransmissions with latency and request backlogs.
- Verify the application accept loop and worker capacity.
- Use connection reuse (keep-alive) where appropriate.
- Tune backlog and buffers only after confirming the real bottleneck.

Production Challenge

Determine whether requests are queueing due to kernel limits, network issues, or a slow application, and fix the root cause without simply increasing queue limits.

Senior Judgment

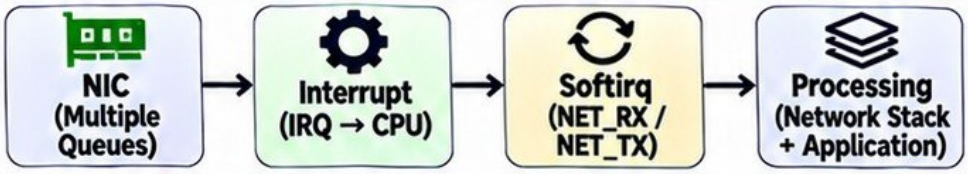
Do not increase kernel queue limits merely to conceal a slow application. Validate the bottleneck and address the application's capacity and I/O behavior.

One CPU Core Is Handling All the Network Traffic

Key Objectives

- Investigate NIC queues, interrupts, softirqs, and uneven packet processing.
- Examine RSS, RPS, RFS, XPS, and IRQ affinity.
- Use /proc/interrupts, /proc/softirqs, ethtool, and mpstat.
- Identify packet drops caused by CPU-local processing bottlenecks.
- Improve packet distribution across cores while keeping overhead low.

Packet Processing Flow (and Where It Can Go Wrong)



Problem: If IRQ/softirq work is stuck on one CPU core, that core becomes a bottleneck, causing high CPU, latency, and drops.

Key Concepts

RSS (Receive Side Scaling)	RPS (Receive Packet Steering)	RFS (Receive Flow Steering)	XPS (Transmit Packet Steering)	IRQ Affinity
Distributes incoming packets across multiple NIC queues/CPUs.	Steers packets to different CPUs in software (when NIC queues are few).	Keeps a flow on the same CPU as the application thread (cache-friendly).	Spreads outgoing packets across CPUs to reduce contention.	Controls which CPU core handles a specific NIC interrupt (IRQ).

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`cat /proc/interrupts`	View interrupt counts per CPU.	NIC IRQs spread across multiple CPUs.
`cat /proc/softirqs`	Check softirq activity (e.g., NET_RX/NET_TX).	NET_RX not concentrated on one CPU.
`ethtool -l eth0`	Show NIC queue count (RX/TX).	Multiple queues (e.g., Combined: 8).
`ethtool -x eth0`	View RSS hash/queue mapping.	Queues mapped to different CPUs.
`mpstat -P ALL 1`	Per-CPU utilization.	One core not abnormally high (e.g., > 80%).
`ethtool -S eth0 | grep -i drop`	Check NIC/driver drop counters.	rx_dropped, rx_missed_errors, etc. (should be low).

Signs of a CPU-Local Bottleneck

- One CPU core has very high usage (irq/softirq).
- NET_RX or NET_TX softirq is concentrated on a single core.
- Increasing packet drops (e.g., rx_dropped).
- High network latency under load, even with available CPU on other cores.

Tuning Actions (Practical)

- Enable/configure RSS (ethtool -L / -X).
- Set IRQ affinity to spread NIC interrupts (e.g., smp_affinity or irqbalance).
- Use RPS/RFS (e.g., sysfs rps_cpus).
- Use XPS for transmit distribution.
- Validate with mpstat, /proc/softirqs, and drop counters after changes.

Production Challenge

Improve packet distribution across CPU cores (RSS/RPS/RFS/XPS and IRQ affinity) to eliminate the single-core bottleneck without adding unnecessary cross-core overhead.

Senior Judgment

Distribute network workload across cores based on hardware capability (NIC queues) and measured behavior, not guesswork.

TCP Tuning That Improves Throughput but Destroys Tail Latency

Key Objectives

- Examine TCP congestion control, receive windows, retransmission behavior, and buffer sizing.
- Compare CUBIC and BBR where supported and appropriate.
- Investigate RTT variation, packet loss, and bufferbloat.
- Use ss -i, tc, and controlled network impairment experiments.
- Select TCP tuning based on workload, network conditions, and latency objectives.

TCP Behavior and the Trade-off

Sender (Send Window + Congestion Control) → Network (RTT, Loss, Queue/Buffer) → Receiver (Receive Window + Buffer) → Outcome (Higher Throughput vs. Higher Tail Latency)

Bigger windows and more aggressive tuning can increase throughput but also build up queue delay, causing higher tail latency.

Key Concepts

Congestion Control
Algorithm that adjusts send rate (e.g., CUBIC, BBR).

Receive Window (RWIN)
How much data the receiver can accept before acknowledging more.

Retransmission
Resends lost packets (driven by loss, RTT, and timeouts).

Bufferbloat
Excessive buffering in the network that increases queue delay and tail latency.

Tail Latency (p99)
The high-end latency (e.g., 99th percentile) that users feel during congestion.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`ss -i`	Inspect TCP socket details.	cwnd, rtt, rttvar, retrans, rcv_space.
`sysctl net.ipv4.tcp_congestion_control`	View/set congestion control algorithm.	cubic (default) or bbr (if supported).
`sysctl net.ipv4.tcp_rmem`	Check receive buffer range (min, default, max).	e.g., 4096 87380 6291456.
`sysctl net.ipv4.tcp_wmem`	Check send buffer range.	e.g., 4096 65536 6291456.
`tc qdisc add .... netem ...`	Create network impairment for testing.	e.g., delay 50ms loss 1% (test behavior).
`ping -c 100 <host>`	Measure RTT and variation.	min/avg/max/mdev (RTT stability).

CUBIC vs BBR (Quick Comparison)

Aspect	CUBIC	BBR
Goal	Loss-based, maximize throughput	Model-based, use bottleneck bandwidth/RTT
Throughput	High (stable, widely used)	High (often higher in long-fat networks)
Tail Latency	Can increase under bufferbloat	Often lower, more stable
Best For	General purpose (default, mature)	High-latency or variable networks (when supported)

Tuning Guidelines

- Choose the right congestion control (CUBIC or BBR) for your network and workload.
- Tune receive/send buffers to match bandwidth and RTT.
- Detect and reduce bufferbloat (e.g., with fq_codel or CAKE).
- Validate with real tests (ss -i, tc netem) and check p50/p99 latency.

Production Challenge

Select and validate TCP tuning that improves throughput while keeping tail latency (p99) within your SLA, based on real workload and network conditions.

Senior Decision

Tune TCP deliberately for your use case, network conditions, and latency targets, and always measure both throughput and tail latency (e.g., p99) before and after changes.

The Application Is Waiting, but Nobody Knows What It Is Waiting For

Key Objectives

- Investigate futex contention, mutex waits, file I/O waits, and syscall latency.
- Use strace -c, perf, and off-CPU profiling techniques.
- Distinguish CPU-bound work from lock contention and blocking operations.
- Analyze why adding worker threads can make throughput worse.
- Identify the dominant source of off-CPU latency in production.

What the Application Might Be Waiting On

Futex / Mutex (Lock Wait) → File I/O (Disk, FS, fsync, etc.) → Syscall Wait (e.g., read, write, connect) → Other Blocking (e.g., network, I/O, resource limits)

The application can spend significant time waiting (off-CPU) for locks, I/O, or system calls, even when CPU usage is low.

Key Concepts

Futex
Fast user-space locking mechanism used by many threading libraries (e.g., pthreads).

Mutex (Lock Wait)
A lock that serializes access to shared resources; can cause contention.

I/O Wait
Time spent waiting for disk or filesystems (e.g., read, write, fsync).

Syscall Latency
Time spent inside system calls (e.g., read(), write(), connect(), futex()).

Off-CPU Profiling
Measures where time is spent waiting (rather than on-CPU executing).

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`strace -c -p <pid>`	Summarize system calls and time spent.	High time in futex, read, write, fsync, etc.
`perf record -g -p <pid>`	Record CPU and off-CPU events.	Identify lock waits, I/O waits, and hotspots.
`perf report`	Analyze recorded data (call graph).	Functions with high wait/off-CPU time.
`sudo perf sched timehist -p <pid>`	View thread scheduling and wait times.	Long sleep/wait periods and blocked threads.
`pidstat -w -p <pid> 1`	Show task waiting (I/O, CPU).	High %wait indicates blocking (I/O or lock).

Common Wait Sources and Signs

Wait Source	Symptoms / Indicators
Futex / Lock	High futex() time, many threads sleeping, low CPU usage.
File I/O	High read/write/fsync time, disk latency spikes.
Syscall Wait	Large syscall time in strace -c (e.g., read, write, connect).
Resource Limits	Threads blocked (e.g., thread pool exhausted, file descriptors).

Investigation Steps

- Check overall CPU vs. wait time (pidstat, top).
- Use strace -c to see long-running system calls.
- Profile with perf (on-CPU and off-CPU).
- Identify lock contention (futex/mutex).
- Correlate with I/O and resource metrics (iostat, sar, lsof).
- Validate with a focused test scenario (reproduce and measure).

Production Challenge

Identify the dominant cause of off-CPU latency (lock contention, I/O, or syscall waits) and fix it without simply adding more worker threads.

Senior Decision

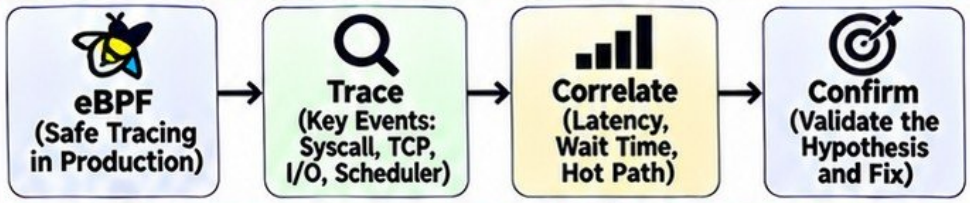
Find the real reason the application is waiting, remove the bottleneck (e.g., lock, I/O, or resource limit), and only then consider scaling worker threads if it is actually beneficial.

eBPF Reveals the Bottleneck That Traditional Monitoring Missed

Key Objectives

- Introduce safe, production-focused tracing with bpftrace and BCC tools.
- Investigate syscall latency, scheduler delays, TCP retransmissions, and block I/O.
- Compare sampling, tracepoints, kprobes, and uprobes.
- Examine tracing overhead, kernel compatibility, and production safety.
- Build and validate a latency hypothesis with targeted tracing.

How eBPF Helps You Find the Real Bottleneck



eBPF gives low-overhead, real-time visibility into what the kernel and your application are actually doing, without broad guessing.

Key Concepts

bpftrace
A simple language for quick, low-overhead tracing.

BCC (BPF Compiler Collection)
A library of ready-to-use eBPF tools (e.g., execsnoop, biolatency, tcptop).

Tracepoint
Built-in kernel events (e.g., sched, block, tcp) with low overhead.

Kprobe / Uprobe
Dynamic tracing of kernel code (kprobe) or user-space functions (uprobe).

Sampling
Captures a representative subset of events to reduce overhead (e.g., profile).

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`bpftrace -l`	List available tracepoints and probes.	`bpftrace -l 'syscalls:*'`
`bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'`	Trace syscall activity (e.g., file opens).	Identify top processes using a syscall.
`sudo biolatency -m 1`	Measure block I/O latency (BCC).	Watch for high I/O latency spikes.
`sudo tcptop`	Show active TCP connections and rates.	Find high traffic or retransmissions.
`sudo runqlat -p <pid>`	Measure scheduler (run-queue) delay.	Check for high scheduling latency.

Tracing Targets and Examples

Target	Use Case (Example)
Syscall Latency	Find slow system calls (e.g., read, write, fsync).
Scheduler Delay	Detect time waiting for CPU (e.g., runqlat, runqslower).
TCP Retransmission	Identify network loss/retries (e.g., tcptop, tcp_retrans).
Block I/O	Measure disk latency and slow I/O (e.g., biolatency).

Production Safety & Overhead

- Start with a clear hypothesis (e.g., "I/O is causing latency").
- Use focused, short-lived tracing (on specific events or a single PID).
- Prefer tracepoints where possible (lower overhead).
- Validate kernel and bpftrace/BCC compatibility (e.g., Linux 5.x+).
- Monitor system impact (CPU, memory) while tracing and stop if it's too high.

Practical Investigation

Build a latency hypothesis, run a targeted eBPF trace (e.g., syscall, I/O, or TCP), and use the results to confirm or reject the root cause, instead of collecting everything.

Senior Decision

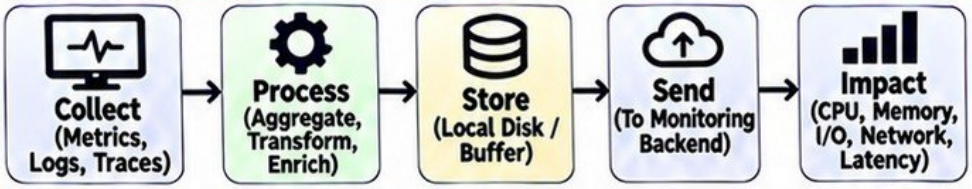
Use eBPF intentionally: start with a hypothesis, trace only the relevant events, validate the bottleneck, and fix with confidence.

Your Monitoring Agent Is Contributing to the Performance Incident

Key Objectives

- Investigate observability overhead, high-cardinality metrics, and excessive log collection.
- Examine profiling costs, frequent polling, and tracing volume.
- Measure agent CPU usage, memory consumption, disk writes, and network traffic.
- Compare always-on instrumentation with targeted diagnostics.
- Make a balanced decision to retain essential visibility without causing saturation.

How a Monitoring Agent Can Cause Performance Issues



Observability brings visibility, but excessive collection, high-cardinality data, or heavy processing can consume real resources and degrade performance.

Key Concepts

Observability Overhead
The resource cost (CPU, memory, I/O, network) of collecting and sending data.

High-Cardinality
Too many unique label combinations (e.g., user_id, request_id) that increase CPU, memory, and storage.

Log Collection
Excessive or verbose logs that drive high disk writes and network traffic.

Profiling Cost
Continuous CPU/memory profiling that adds overhead (e.g., eBPF, perf, or runtime profilers).

Tracing Volume
Large numbers of spans and high sampling rates that increase CPU, memory, and backend load.

Essential Commands and What to Look For

Command	Purpose	What to Check / Example
`top / htop`	Check CPU and memory usage.	Is the monitoring agent high in CPU/RAM?
`pidstat -u -r -d -p <pid> 1`	Measure CPU, memory, and disk I/O.	e.g., `pidstat -u -r -d -p 1234` (agent PID).
`iotop -oP`	See real-time disk write activity.	High write rate from the agent (e.g., MB/s).
`iftop / nethogs`	Check network traffic per process.	High outbound traffic to metrics/log backend.
`du -sh /var/log/*`	Check log volume and growth.	Large or fast-growing log files from the agent.

Common Sources of Overhead

Source	Example of Impact
Metrics (High Cardinality)	Large number of unique labels (e.g., per-user, per-request) causes high CPU and memory.
Log Collection	Verbose logs or collecting unnecessary logs increases disk writes and network usage.
Profiling	Continuous profiling (CPU/heap) adds measurable CPU overhead.
Tracing	High sampling rate or large trace volume increases CPU, memory, and egress traffic.

Tuning Actions

- Limit metric cardinality (remove unnecessary labels).
- Reduce log verbosity and apply log sampling/retention.
- Lower profiling frequency or use on-demand profiling.
- Adjust tracing sampling rate (e.g., 1–10%) based on need.
- Set resource limits for the agent (e.g., CPU and memory cgroups).

Production Challenge

Maintain enough observability to investigate and resolve incidents while preventing the monitoring agent from becoming another source of saturation.

Senior Judgment

Keep observability purposeful and cost-aware: collect what you need, at the right frequency and volume, and always measure the agent's own impact on your system.

The sysctl Change That Fixed One Workload and Broke Three Others

Key Objectives

- Investigate kernel tuning changes affecting networking, memory, and file descriptors.
- Compare runtime settings with persistent configuration under /etc/sysctl.d/.
- Examine workload-specific benefits and system-wide side effects.
- Design controlled experiments with baselines, rollback plans, and acceptance criteria.
- Reject a change that improves average throughput but violates tail-latency objectives.

How a sysctl Change Can Help One Workload and Harm Others

Change sysctl (Kernel Tuning) → Improves One Workload (e.g., higher throughput) → Side Effects on Others (e.g., higher latency, more memory use) → System Impact (Conflicting workloads, SLA breach)

Kernel tuning is system-wide, so a change that helps one workload can shift resources, increase contention, or hurt the tail latency of others.

Key Concepts

sysctl (Kernel Tuning)
Runtime and persistent kernel settings (e.g., network, memory, FD limits).

Runtime vs Persistent
Runtime: "sysctl -w" (temporary). Persistent: "/etc/sysctl.d/" (survives reboot).

Tuning Areas
Networking (e.g., buffers, backlog), Memory (e.g., swappiness), File Descriptors (e.g., fs.file-max).

Controlled Experiment
Change one variable, keep baseline, measure impact, and have a rollback plan.

Tail Latency (p99/p999)
Worst-case latency experienced by users, not just the average throughput.

Common sysctl Examples and Their Trade-offs

sysctl Setting	Area	Potential Benefit	Possible Side Effect
net.core.somaxconn	Networking	Handles more pending connections.	Higher memory usage; can hide a slow app.
net.ipv4.tcp_rmem / tcp_wmem	Networking	Higher throughput on high-BDP links.	More memory per connection; pressure under load.
vm.swappiness	Memory	Keeps more memory for applications.	Risk of OOM if set too low.
fs.file-max	File Descriptors	Supports more open files (high concurrency).	Higher kernel memory usage.

Experiment and Change Process

1. **Baseline:** Measure current performance (e.g., throughput, p95/p99 latency, errors).
2. **Change:** Apply one sysctl change (e.g., "sysctl -w ...") in a controlled test.
3. **Validate:** Compare results across all key workloads (not just the target workload).
4. **Decide:** Keep, adjust, or rollback (e.g., "sysctl -w <old_value>" or restore file).

Safety and Rollback Plan

- Test in a staging environment with real or representative load.
- Change one parameter at a time.
- Define acceptance criteria (e.g., p99 latency ≤ target, error rate ≤ threshold).
- Have a rollback plan (previous value and command ready).
- Document the change and its impact for future reference.

Production Challenge

Reject a kernel tuning change if it improves average throughput but causes higher tail latency (p99/p999) or negatively impacts other critical workloads and SLAs.

Senior Judgment

Kernel tuning is a trade-off: optimize for overall system and business objectives, not just a single workload metric.

The Performance Fix Passed Benchmarking and Failed in Production

Key Objectives

- Investigate why a change performed well in benchmarks but failed in production.
- Examine warm caches, unrealistic traffic, and coordinated omission.
- Compare benchmark tools (fio, stress-ng, sysbench) and workload-specific tests.
- Assess reproducibility, environment differences, and statistical variability.
- Define key metrics (throughput, p50, p95, p99, error rate, saturation).

Why Benchmarks Can Be Misleading

Benchmark Setup (Often Ideal) → **Hides Real Conditions** (e.g., warm cache, low concurrency) → **Passes In Testing** (High throughput) → **Fails in Production** (e.g., higher latency, errors, resource contention)

Benchmarks can look good while ignoring real-world conditions such as cache state, traffic patterns, contention, and longer runtimes.

Key Concepts

Warm Caches
Data already in memory, which hides real disk I/O latency.

Unrealistic Traffic
Test traffic that does not match real production patterns (e.g., concurrency, request mix).

Coordinated Omission
A testing flaw that hides latency spikes by not accounting for delayed requests.

Statistical Variability
Performance can vary between runs; use multiple runs and confidence ranges.

Saturation Guardrail
A safe limit (e.g., ≤ 70–80% CPU, I/O, or network) to avoid operating at the edge of failure.

Eenchmarking Tools and When to Use Them

Tool	Primary Use	Example Command	Notes
fio	Disk and I/O performance	fio --name=rand --rw=randrw --bs=4k --iodepth=32	Flexible, realistic I/O patterns.
stress-ng	System-wide resource stress	stress-ng --cpu 4 --io 2 --vm 2 --timeout 60s	Test CPU, memory, I/O, and more.
sysbench	CPU, memory, and database	sysbench cpu run --threads=8 --time=60	Good for CPU and DB-like workloads.
Workload Test	Your real application behavior	e.g., k6, wrk, or application-specific script	Most reliable for production-like results.

Key Metrics (with Meaning)

Metric	Meaning
Throughput	Amount of work per unit time (e.g., requests/sec, IOPS).
p50 (Median)	Typical response time (50% of requests are faster).
p95	95% of requests complete within this time.
p99	99% of requests complete within this time.
Error Rate	Percentage of failed requests (e.g., timeouts, 5xx).
Saturation	Resource usage level (e.g., CPU, I/O, network), keep below a safe threshold (e.g., ≤ 70–80%).

A Reliable Testing Approach

- Test with cold and warm caches (to see both disk and memory behavior).
- Use production-like traffic (concurrency, request mix, and data size).
- Run for a sufficient duration (e.g., 15–60+ minutes) to catch stability issues.
- Repeat tests and compare results (average, p95, and p99 with variation).
- Set acceptance criteria (e.g., throughput ≥ target, p99 ≤ X ms, error rate ≤ Y%).

Production Decision

Approve a change only when representative testing (realistic workload, duration, and metrics) confirms it will meet production objectives for throughput, latency (p50/p95/p99), error rate, and stability.

Senior Decision

Trust representative, reproducible, and statistically sound testing, not just a fast benchmark result.

Final Production Incident. Defend Your Linux Performance Tuning Decisions

Incident Scenario

A Linux-hosted API experiences:

- Rising p99 latency
- Intermittent timeouts
- Database stalls
- Occasional container restarts.

Structured Investigation Workflow

Collect Evidence (Metrics, Logs, Traces) → Analyze (Find Patterns and Anomalies) → Identify Root Cause (Primary & Secondary) → Tune & Mitigate (Immediate, Then Permanent) → Verify (Measure, Compare, Confirm)

Use a structured, evidence-driven process to find the real bottleneck, apply safe tuning, and verify the improvement.

Evidence to Analyze

- CPU run queues (e.g., load average, runq)
- Memory PSI (/proc/pressure/memory)
- cgroup throttling (e.g., CPU quota/throttle)
- TCP retransmissions (e.g., ss, nstat)
- Block I/O latency (e.g., iostat, blktrace)
- Application traces (e.g., p95/p99, spans).

Key Analysis: What to Determine

Primary Bottleneck	Secondary Effects	Misleading Metrics	Mitigation (Immediate)	Remediation (Permanent)
The main limiter (e.g., CPU contention, I/O latency, memory pressure, or database I/O).	Knock-on issues (e.g., cgroup throttling, TCP retransmissions, queue buildup, restarts).	Signals that can look like the cause (e.g., high CPU from I/O wait, restarts from OOM).	Quick, low-risk actions to stabilize the system (e.g., adjust limits, reduce contention).	Sustainable fixes (e.g., right-sizing resources, proper tuning, code or query optimization).

Investigation Commands (Examples)

Area	Key Commands
CPU & Run Queue	`uptime; top; mpstat -P ALL 1; vmstat 1`
Memory & PSI	`free -h; cat /proc/pressure/memory; vmstat -s`
cgroup Throttling	`cat /sys/fs/cgroup/*/cpu.stat; kubectl top pod`
TCP / Network	`ss -tinm; nstat -az; ip -s link`

Area	Key Commands
Block I/O	`iostat -x 1; pidstat -d 1; blktrace (targeted)`
Database	`pg_stat_activity / slow query log (or equivalent)`
App Traces	`curl -s http://..../metrics; trace query (e.g., Jaeger, Tempo, OpenTelemetry)`
Containers	`kubectl describe pod; kubectl logs; docker stats`

Performance Report Deliverable

- Incident summary (impact, timeline).
- Evidence and analysis (commands + outputs).
- Root cause (primary) and secondary effects.
- Tuning decisions (what, why, and risks).
- Before-and-after measurements (e.g., p50/p95/p99, throughput, error rate).
- Verification results (stability and SLOs).

Rollback Procedure

- Keep a pre-change baseline.
- Document all changes (e.g., sysctl, limits, resources).
- Use versioned configs (e.g., /etc/sysctl.d/, IaC).
- Revert to the previous config (one command or commit).
- Validate rollback (metrics return to baseline, no new errors).

Acceptance & Verification

- p99 latency back within target (e.g., ≤ 200 ms).
- Error rate within SLO (e.g., ≤ 1%).
- No recurring timeouts or restarts.
- System stable for a defined period (e.g., ≥ 24–48 hours).
- Document and close with lessons learned and next actions.

Senior Decision

Approve the tuning only when the evidence clearly identifies the root cause, the change is safe, improves p99 and stability, and has a tested rollback.